

• **Assignment Overview**

This assignment forms the core of Coursework One and is designed to simulate a real-world investment strategy development process. Working in teams, students will design and implement the data extraction, processing, and storage pipeline for structured or unstructured data that will later feed into Coursework Two, where the investment strategy itself will be implemented.

The key objective of this coursework is to deliver a rational, efficient, and well-documented data storage design that enables the easy retrieval of investment indicators by company or by year.

Goals

The goal of Coursework One is to design, develop and implement a functional data product based on data extractions for companies listed in the Equity Database (Investable Universe) and design the its storage in order to facilitate data retrieval.

By completing this coursework, students will:

1. Collaborate effectively in a team to deliver a coherent data product.
2. Apply data engineering and product development principles in a professional team context.
3. Gain hands-on experience in functional roles, including data product owner, product specialist, and developer.
4. Understand the full lifecycle of a data product — from conceptualization and planning to implementation and validation.
5. Produce documentation and artefacts that meet professional and academic standards for data-driven projects.

Team Structure

Each team will consist of up to 7 students, distributed among the following roles:

- Investment Product Owners (Portfolio Managers) – oversee the project direction and define business requirements.
- Portfolio Product Specialists – translate investment needs into data specifications and align output with Coursework Two requirements.
- Developers (Data and Software Engineers) – design, implement, and test the ETL and storage systems.

Teams are free to decide how to assign members to each role and how to balance responsibilities.

A finalized team structure must be submitted by Friday, 13 February 2026, via email to the Professor and Teaching Assistant.

Infrastructure Design: Minimum Specifications

For each company stored in the PostgreSQL database `fit`, within the schema `systematic_equity` and the table `company_static`, you are required to design a solution that addresses the following aspects:

9. Identify the data needed to build a factor that will be used in Coursework Two for portfolio construction.
10. Ingest the data from the source.
11. Persistently store the data in a structured manner.

In developing this solution, it is essential to implement a flexible design capable of addressing the following challenges:

12. Handling the addition or removal of companies from the `company_static` table.
13. Retrieving past data (at least 5 years).
14. Ensuring that the solution can be run regularly (daily, weekly, monthly, or quarterly) depending on the strategy goals.

Infrastructure Design

The infrastructure for this project will be centered around a comprehensive data lake, which will serve as the foundation for future analytics in Coursework Two. You can identify the best pattern for storing the reports and this design aims to provide a scalable, efficient, and flexible storage solution for the data.

Data Lake Implementation Coursework One

The core of the infrastructure will be a data lake. If a file system solution is chosen this needs to be implemented using MinIO.

- Data Ingestion and Processing Basic Tools
 - To handle data ingestion and processing, you can leverage on Apache Kafka.
 - Database Systems: Two database systems are provided by default MongoDB & PostgreSQL.
 - Object Data Storage: MinIO data storage.

In order to set-up the tools needed, please cd in the root directory of this repo and execute the command:

```
docker compose up --build <service-needed> ...
```

All databases and softwares will be set up in docker container and will be exposed as per [docker-compose.yml](#) specs. Please, read carefully docker compose file to identify the specification of the software needed.

Data specialists and developers are encouraged to create appropriate schemas, tables, and collections in these databases as needed to support the project goals.

Additional Components

Developers and specialist can expand on the technologies provided however, any additional components required for the infrastructure must be containerized using Docker. This includes, but is not limited to:

- Data quality and validation tools
- Metadata management systems
- Security and access control services

Code Submission

Code must be submitted to github by Friday 27/02/2026 at 1400 GMT.

For each coursework submission code must be submitted to github via pull request assigned to @uceslc0 for final review before merging. The repository for submission is https://github.com/iftucl/ift_coursework_2025.

In order to submit the coursework, the students must to follow the following steps:

19. Fork the repository;
20. create a new branch;
21. add your own developments in a dedicated folder;
22. the dedicated folder has to be placed under "./ift_coursework_2025/" and is structured as following:

xxxxxxxxxx

team_<insert your team id>

```
|—— CHANGELOG.md
|—— coursework_one/
|   |—— .gitkeep
|   |—— config/
|       |—— conf.yaml
|—— modules/
|—— static/
|—— test/
|—— Main.*
|—— project.toml
```

```
|—— .gitkeep
|—— README.md
|—— coursework_two/
|   |—— .gitkeep
```

Subfolder ./modules can be further structured in sub-folders name after what the contain. an example could be:

```
|—— modules/
|   |—— db/
|       |—— db_connection.*
|   |—— input/
|       |—— input_loader.*
|   |—— output/
|       |—— script_purposes.*
|       |—— etc..etc..
```

Do not copy databases in other folders. There is one source only for databases and this is in folder 000.Database.

In addition, un-stage any change to 000.Database folder before committing to Git. Any change outside your group folder committed to git will make the pull request invalid.

Python Specifications

Package Management Use Poetry as the package manager for your Python project:

- Initialize your project with `poetry init`
- Manage dependencies using `poetry add` and `poetry remove`
- Ensure your `pyproject.toml` file is well-maintained and includes all necessary dependencies

Application Flexibility Design the application to be flexible in its execution frequency:

- Implement command-line arguments or configuration files to specify run frequency (daily, weekly, or monthly) or run date
- Use scheduling libraries like APScheduler or Airflow for more complex scheduling needs
- Ensure all time-sensitive operations are parameterized to accommodate different run frequencies

Testing

Implement comprehensive testing for your application:

- Write unit tests for individual functions and methods
- Develop integration tests for interactions between different components
- Create end-to-end tests to verify the entire data pipeline
- Aim for a minimum of 80% test coverage
- You must use pytest as the testing framework through poetry command like `poetry run pytest ./tests/`

Code Quality

Ensure code quality through linting and formatting:

- Use *flake8* for linting Python code
- Implement *black* for consistent code formatting
- Configure and use isort for import sorting

Security

Conduct regular vulnerability scans:

- Use tools like Bandit or Safety to scan for known vulnerabilities in your dependencies
- Implement and maintain a process for addressing identified vulnerabilities promptly

Documentation Provide comprehensive documentation using Sphinx:

- Use docstrings for all modules, classes, and functions following the Sphinx notation
- Create a docs directory in your project root for Sphinx documentation

Include the following in your documentation:

- Installation guide
- Usage instructions
- API reference
- Architecture overview
- Generate and maintain up-to-date HTML documentation

Appendix 1. Responsibilities of Investment Product Owners

The Investment Product Owners (Investment Managers) coordinate and oversee the activity of the wider team, acting as the bridge between the investment vision and the technical implementation. They ensure that the data pipeline supports the long-term strategic goals of investment product design and portfolio construction.

- Strategic Leadership
 - Based on the investable universe, define and communicate the investment product vision — articulate the overarching purpose, target audience, and unique value proposition of the proposed investment strategy.
 - Develop the product roadmap — outline what the product aims to achieve, why it is innovative, the expected benefits, potential costs, and prospective users or clients (e.g., a fund manager, a retail investor, an ESG-focused portfolio).
 - Align business and technical objectives — ensure that the dataset and its design contribute to measurable, investable factors consistent with the intended investment strategy in Coursework Two.
 - Monitor project progress — maintain oversight of milestones, timelines, and deliverable quality.
- Prepare and deliver a compelling presentation of the final product

- Demonstrate how the product meets its intended goals and requirements
- Conduct market research and competitive analysis
- Stay informed about industry trends and competitor offerings
- Incorporate insights into product strategy
- Analytical Direction
 - Specify investment hypotheses and drivers — determine the economic or market logic that underpins the strategy (e.g., valuation factors, momentum effects, ESG signals).
 - Identify key indicators for portfolio construction — define the investment factors to be modelled and describe how they will be used in portfolio allocation or ranking.
 - Determine data requirements — establish what raw data are needed to compute these indicators, their sources, and the relevant frequency (annual, quarterly, daily).
- Collaboration and Communication
 - Interface with Product Specialists to translate the investment vision into precise data specifications and ensure analytical soundness.
 - Coordinate with Developers to guarantee that data pipelines, transformations, and storage architectures enable the intended analyses and factor calculations.
 - Facilitate decision-making and documentation — lead team discussions, record rationale behind design choices, and ensure coherence of all outputs.
- Governance and Quality Control
 - Oversee data ethics and compliance — ensure that the use and sharing of data adhere to university policy and responsible data management practices.
 - Maintain traceability of decisions and data sources through version-controlled documentation.
 - Approve final deliverables — validate that the submitted prototype, storage design, and reports reflect the agreed investment logic and quality standards.

Appendix 2. Responsibilities of Investment Specialists

Data specialist act as a bridge between investment product owners and developers. They are responsible to identify and design the technical implementation of the data product

- Define and prioritize strategy requirements

- Create and maintain a a log of all data needed to build the factor(s)
 - Prioritize features based on value and feasibility
- Facilitate communication between technical and non-technical team members
 - Translate business requirements into technical specifications using [issues](#)
 - When creating a new issue, you will use the team label created for your team in github
 - Assign and monitor issue progress
 - Identify and design the technical implementation of the data product
- Analyze business requirements and propose suitable technical solutions
 - Create high-level system architecture diagrams ([draw.io](#) or [lucidcharts](#))
 - Define and maintain technical implementation documentation
- Provide an overview of the product's purpose and its advantages over alternatives
 - Conduct and document research on available technologies, including:
 - Potential solutions for delivering the final product
 - Alternatives to chosen technologies
 - Pros and cons of each option
- Create and update detailed design documentation for the data product
 - Own data governance artifacts
 - Develop and maintain data catalogs
 - Create and update data dictionaries
 - Design and document data lineages
- Ensure the technical feasibility of product requirements
 - Assess the viability of proposed features from a technical standpoint
 - Provide feedback to product owners on potential technical challenges or limitations
 - Write user acceptance criteria and detailed requirements
 - Develop clear, testable acceptance criteria for each feature
- Collaborate with product owners to refine and clarify requirements
- Support the development team with constant meetings and daily stand-ups to review requirements.
- Provide technical guidance and clarification on requirements
- Contribute to quality assurance
- Define data quality standards and metrics
- Collaborate with developers to implement data validation processes

Appendix 3. Responsibilities of the Developers

- Implement the data product based on requirements, specifications and issues
- Develop data extraction, processing, and analysis pipelines
- Propose and implement appropriate infrastructure and technology solutions
- Collaborate with data specialists to refine technical requirements
- Conduct code reviews and ensure code quality
- Infrastructure Design
- Code Documentation

Appendix 4. How to Submit Code

The repository for submission is on [github](#). In order to submit the coursework, students must follow these steps:

70. Fork the Repository:
71. Navigate to the GitHub page of the repository.
72. Click on the Fork button at the top right corner of the page to create a copy of the repository in your GitHub account.
73. Clone Your Forked Repository

```
git clone https://github.com/YOUR-USERNAME/ift_coursework_2025.git
```

Create a New Branch:

Change into your cloned repository directory:

```
xxxxxxxxxx
```

```
cd ift_coursework_2025
```

Create and switch to a new branch:

```
git checkout -b feature/coursework_one_YOUR_TEAM_ID
```

From here you can now add all your developments and commit changes.

Please note: Since multiple developers may be working on this project, your team can create and work on multiple branches during development. However, for the final submission, only one branch and one pull request will be accepted. Before submitting, ensure that all development branches are merged into a single branch named `feature/coursework_one_YOUR_TEAM_ID`. This branch should then be used to create the pull request for submission.

Open a Pull Request:

When you are ready for submitting your code, go to your forked repository on GitHub.

74. Click on the Pull Requests tab.
75. Click on New Pull Request.
76. Select main from the original repository as the base and your new branch from your fork as the compare.
77. Assign Luca Cocconcelli for review and submit your pull request.

Appendix 5. Marking criteria

The marking criteria of this coursework is as follows:

- design of an efficient technical implementation (30%).
- flexible and professional code implementation correctly submitted to github (40%).
- technical documentation of infrastructure, architecture and implementation (30%) report must be submitted to UCL Turnitin (20,000 words max).

Final report should contain the following sections:

81. introduction
82. investment goals and needs from a data perspective
83. proposed solution & vision: why, what and how this solution is implemented
84. architecture and infrastructure design
85. conclusions